

Vivadh - Technical Briefing & Project Walkthrough

Introduction

Core Purpose: Vivadh is an AI-powered multidimensional debate platform.

Problem Solved: It enables users to orchestrate autonomous, multi-agent debates on any chosen topic. Users can assign distinct, conflicting personas (e.g., *Proposition*, *Devil's Advocate*, *Socratic*) to different Large Language Models (LLMs) and watch them structurally argue, counter-argue, and interact with each other in real-time, providing deep insights into complex topics natively unachievable by a solo AI chatbot.

Requirements

Functional Requirements:

- **Multi-Agent Orchestration:** Support orchestration of up to 4 distinct AI models (Llama 3.3, Llama 3.1, Llama 4 Scout, Qwen/DeepSeek via Groq).
- **Real-Time Token Streaming:** Stream AI responses fluidly using Server-Sent Events (SSE).
- **Turn-based Interaction:** Enable multiple rounds and allow the user to step in dynamically as a *Moderator*.
- **Session State Management:** Debate history preservation, retrieval, and automated post-debate summarization.
- **Authentication:** User authentication and authorization (utilizing Firebase).

Non-Functional Requirements:

- **High-Speed Inference:** Use of Groq LPU API to ensure latency of sequential LLM calls remains low enough for a fluid user experience.
- **Modern UI/UX:** Responsive, dark-themed interface built using React, Vite, and Tailwind CSS.
- **Scalable Data Schema:** Document-oriented NoSQL storage (MongoDB) for unstructured, append-heavy chat transcripts.

Design Details

- **High-Level Architecture:** Classic Client-Server model. A Vite/React frontend communicates with a scalable Node/Express backend. State and transcript history are persisted in MongoDB Atlas, while Auth is bridged with Firebase.
- **Data Flow (The Streaming Loop):**
 1. The user creates a debate via a REST `POST` request.
 2. The client triggers a round. The Express backend fetches the debate context and calls the Groq API for the first participating model.
 3. The LLM's response is streamed back to the client via `res.write()` (SSE protocol) chunk by chunk.
 4. Once complete, the backend appends the message to the context and repeats the process for the next model in the round *sequentially* (so models can react to what was just said seconds prior).
- **Specific Design Patterns:**
 - **Adapter Pattern:** The generic `callGroqModel` wrapper abstracts the underlying Groq OpenAI-compatible SDK streaming complexities, making it easy to swap inference providers later.
 - **Strategy Pattern (Prompt Engineering):** The `buildSystemPrompt` function dynamically injects conversational roles and tone parameters into the system instructions, radically changing how the model behaves without changing core logic.

Implementation: The "Heart of the Code"

The most critical files driving the core logic are:

1. `server/controllers/debateController.js`
 - **Why it's vital:** This is the engine of the application. The `runRound` function handles the sequential iteration over participating models. It builds the dynamic context block containing both previous round history and current round messages to simulate a fluent, continuous argument. It securely pipes the stream from Groq back to the HTTP response object.
2. `client/src/pages/Debate.jsx`
 - **Why it's vital:** Contains the complex logic to decode and render the SSE stream on the frontend. The `runRound` client handler uses a `TextDecoder` to read `readableStream` chunks precisely, parsing out `data: blocks` and buffering them safely ensuring the UI typewriter effect remains completely sync'd with the server's multi-agent transitions (`model_start -> token -> model_end`).
3. `server/models/Debate.js`
 - **Why it's vital:** Represents the single source of truth for the entire application state. It records the configuration (topic, tone, rounds, roles) and maps a nested document array (`messageSchema`) to persist the transcript, meaning dropped connections don't ruin the debate loop—results are strongly typed and stored.

Interview Deep-Dive

Trade-offs & Technical Debt

- **Sequential vs Concurrent LLM execution:** In `runRound`, models are intentionally iterated over sequentially using `await` inside a `for...of` loop.
 - **Trade-off:** This significantly increases the total round generation time.
 - **Reasoning:** It is a necessary architectural tax. For models to actually debate *each other*, Model 2 must read Model 1's generation from the exact same round.
- **Markdown Handling:** The backend eagerly strips markdown formatting on the *final* string but leaves raw markdown during the live SSE stream. This can lead to minor visual jumping in the UI (e.g. streaming `**bold**` then switching to `bold`). Offloading markdown parsing fully to a frontend library would be much more robust.

Scalability (Handling 10x Load)

If Vivadh receives 10x the concurrent debates, the current HTTP Server-Sent Events implementation poses a heavy risk.

- **The Bottleneck:** Node.js can handle many concurrent connections, but keeping long-running SSE HTTP connections open while waiting sequentially on 4 API

calls per round will exhaust memory and file descriptors under heavy load. Furthermore, Groq API rate-limits will be severely hit.

- **The Solution:**

1. Migrate from long-polling/SSE to **WebSockets** (e.g., Socket.io) for multiplexed, lightweight connections.
2. Decouple the LLM orchestration into a worker queue (e.g., Redis + BullMQ). The Express server would simply enqueue a "Round Requested" event, and external containerized workers would handle the looping Groq API logic and push events to the WebSocket server.

Challenges: Explaining Complex Logic Simply

The Challenge: Decoding a streaming response dynamically in React.

Simple Explanation: Imagine receiving a shattered jigsaw puzzle through a mail slot one piece at a time. The browser's standard `fetch` waits until the *entire* box is delivered. To achieve the "typing" effect, we use a `TextDecoder`. It grabs puzzle pieces (bytes) as they fall through the slot, translates them into text, and glues them together into complete sentences (`\n` separated JSON objects) on the fly, immediately updating the screen so the user doesn't have to wait 10 seconds for the models to finish thinking.

Conclusion

Project Impact: Vivadh demonstrates highly structured usage of Generative AI. It pivots away from standard "User-to-Bot" interactions, showcasing successful "Bot-to-Bot" orchestration. This stands out as a powerful utility for adversarial testing, decision balancing, and educational deep-dives.

Future Enhancements:

- Implementing a real-time asynchronous queue for scalable background inference.
- Branching debate paths (letting the user rewind a round and intervene differently).
- Introducing a "Fact-Checker" model that asynchronously verifies claims made by the debaters.